



T.O.C [Note] - its a new syllabus notes of TOC

Theory of Computation (Pokhara University)



Scan to open on Studocu

Root techniques

Date: _____
Page: _____

1) Mathematical Techniques

The principle of mathematical induction states that only set of natural number containing zero, and with the property that it contains $n+1$ whenever ~~natural no~~ contains all numbers up to end including n , must in fact be the set of all natural number.

It has three steps

- (a) Basic step. First we show that property P is true for 0
 - (b) Induction Hypothesis Assume that P holds for $0, 1, 2, \dots, n$.
 - (c) Induction step: using the induction hypothesis show that P is true for $n+1$.
- Then by the induction principle, P is true for all natural numbers

Example

Prove that the sum of first n natural number is given by

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}, n \geq 1 \text{ by}$$

mathematical induction

Proof :- We can use mathematical induction as follow

(a) Basic step for $n=1$, $LHS = 1$ & $RHS = \frac{1(1+1)}{2}$
 $LHS = RHS = 1$ $= 1$

This implies that, basic step is true for $n=1$

(b) Inductive hypothesis

for $n=k$ given statement becomes
 $1+2+3+\dots+k = \frac{k(k+1)}{2}$

(c) Induction step

for $n=k+1$, given statement becomes
 $1+2+3+\dots+k+k+1$
 $= \frac{k(k+1)}{2} + k+1$
 $= \frac{k(k+1) + 2(k+1)}{2} = \frac{(k+1)(k+1)}{2}$

This shows that the given statement is true for $n=k+1$

hence, $1+2+3+\dots+n = \frac{n(n+1)}{2}$ proved $\#$

Example 2 prove that sum of first n positive odd number $1+3+5+\dots+2n-1=n^2$

(2) The Pigeonhole Principle

If S_1 & S_2 are non-empty finite sets and $|S_1| > |S_2|$, then there is no one-to-one function from S_1 to S_2 (i.e. there is no bijection)

Let A and B be two finite sets with $|A| > |B|$. Then one to one function can not exist from A to B .

(3) The Diagonalization principle

Consider set S_i with numbers $k \in \mathbb{N}$ as initialized below.

$S_i \backslash k$	0	1	2	3
S_0	.		.	.
S_1				
S_2	.	.		
S_3			.	.

Now, we define the diagonal principle as follows:-

Let R be a binary relation on a set S ; let D the diagonal set of R , be

$\{x: x \in S \text{ and } (x, x) \notin R\}$ for each $a \in S$; let $R_n = \{b: b \in S \text{ and } (n, b) \in R\}$, then D is distinct from each R_n .

$$R_0 = \{0, 2, 3\}$$

$$R_1 = \{3\}$$

$$R_2 = \{0, 1, 3\}$$

$$R_3 = \{3\}$$

(4) Proof By Contradiction

Prove that sum of rational and irrational number is irrational by using proof by contradiction technique

Proof:-

We have to show that "sum of rational and irrational number is irrational."

Let ' r ' and ' x ' be two rational and irrational numbers respectively.

We have to prove $(r+x)$ is irrational

Assume that $(r+x)$ is rational. We know that if r is rational, then $-r$ is also rational. And sum of two rational number is always rational.

Then we can write:-

$$(r+x) + (-r) \Rightarrow \text{rational}$$

$$r + x - r \rightarrow \text{rational}$$

$$x \rightarrow \text{rational}$$

Which contradicts that our original assumption

that x is irrational. Hence, sum of rational and irrational number is irrational proved.

5. Doretailing

We can show that union of finite number of countably infinite set is countably infinite.

Let us take these three countably infinite sets

A, B and C as:-

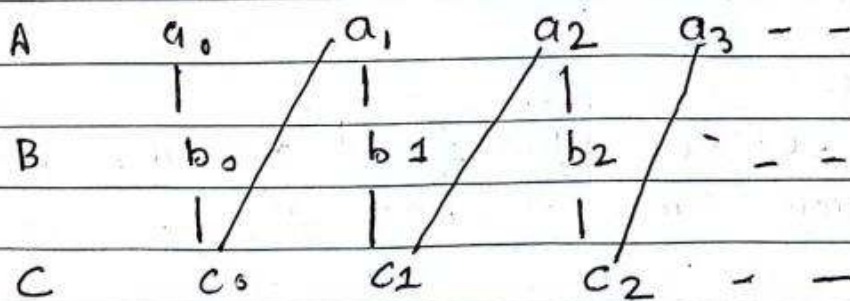
$$A = \{a_0, a_1, a_2, \dots\}$$

$$B = \{b_0, b_1, b_2, \dots\}$$

$$\text{Now } A \cup B \cup C = \{a_0, b_0, c_0, a_1, b_1, c_1, \dots\}$$

In doretailing, we visit first elements of first set first element of second set, first element of third set and so on.

It can be visualize as



hence, doretailing can be defined as technique of interleaving the enumeration of several sets

concatenation of string

concatenation of two strings x and y is denoted by $x.y$ or xy and is given by:

If $x = 010$, $y = 101$ then, $x.y = 010101$

* Substring

A string v is said to be substring of a string w if $w = xy$

- Both x & y called be ϵ

- A string is substring of itself.

* Reversal of string

let w be any string then its reversed is denoted by w^R let $w = baab$ then $w^R = baab$.

Language:

A set of strings all of which are taken from some alphabet Σ^* is called language.

$$L \subseteq \Sigma^*$$

Eg:- $L = \{w \in \{a, b\}^* : w \text{ has odd number of } a\}$

$$L = \{a, ab, abaa, aaaa, abaab, \dots\}$$

Eg- The language of all strings consisting of n 0's followed by n 1's, for some $n \geq 0$:

$L = \{ w \in \{0,1\}^* : w \text{ has } n \text{ 0's followed by } n \text{ 1's and } n \geq 0$

$L = \{ \epsilon, 01, 0011, 000111, \dots \}$

Note: Σ^* is a language for any alphabet Σ .

- $\emptyset$, the empty language, is a language over any alphabet.
- $\{\epsilon\}$, the language consisting of only the empty string, is also a language over alphabet.
- $\emptyset \neq \{\epsilon\}$, the former has no strings and later has, one string.

The function L is defined as follows

- $L(\emptyset) = \emptyset$ and $L(a) = \{a\}$ for each $a \in \Sigma$
- If α and β are regular expressions, then $L(\alpha \cup \beta) = L(\alpha) \cup L(\beta)$
- If α and β are regular expressions, then $L(\alpha\beta) = L(\alpha)L(\beta)$
- If α is a regular expression, then $L(\alpha^*) = \overline{L(\alpha)^*}$ Kleen closure

What language is represented by

$RE \cdot L((a \cup b)^* a)$

$$\begin{aligned}
 &= L((a \cup b)^* \{a\}) \\
 &= (L(a) \cup L(b))^* \{a\} \\
 &= (\{a\} \cup \{b\})^* \{a\} \\
 &= \{a, b\}^* \{a\} = \{w \in \{a, b\}^* \mid w \text{ ends with } a\}
 \end{aligned}$$

Regular Expression

Regular expression is a formula representing language in terms of a form that uses three operations: Concatenation with and Kleen closure. The expression is regular if:

$\emptyset$ and each members of Σ are regular expression.

If α and β are regular expression then so is $(\alpha\beta)$

If α and β are regular expression then so is $(\alpha \cup \beta)$

If α is regular expression then so is α^*
 nothing is else regular expression and the language they represented is established by the use of the function L .

$$L(\emptyset) = \emptyset \quad \text{and} \quad L(a) = \{a\} \quad \text{for each } a \text{ in } \Sigma$$

$$L((\alpha \cup \beta)) = L(\alpha) \cup L(\beta)$$

$$L((\alpha\beta)) = L(\alpha) \cdot L(\beta)$$

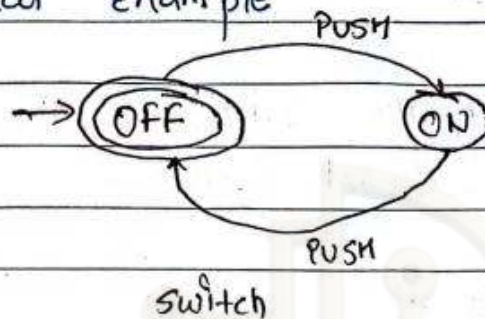
$$L(\alpha)^* = L(\alpha)^*$$

Finite Automation and Register language

Finite Automation

A finite automation has finite set of states / edges lead from one state to another and each edge is labeled with a symbol

Practical Example



some uses:-

- software for designing and checking behavior of digital circuits
- ~~lexical~~ lexical analyzer of typical complex
- for scanning large batteries of texts such as collections of webpages
- software for verifying systems.

Block Diagram

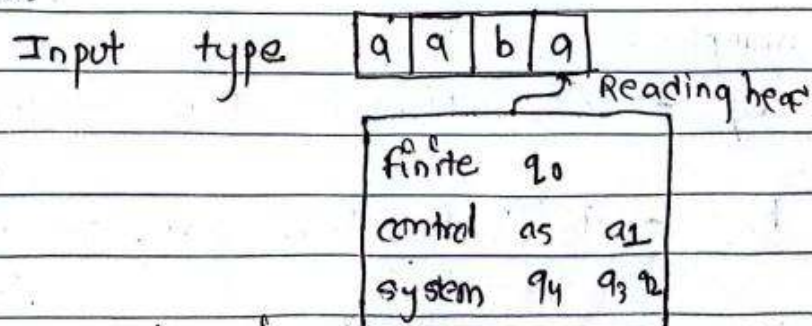


Figure: A finite automation q₀, q₁, q₂, q₃, q₄ are

states in finite control system and
a & b we input symbols.

Deterministic Finite Automaton (DFA)

Formal Definition

A Deterministic finite automaton is a quintuple
 $M = (Q, \Sigma, \delta, q_0, F)$

where, Q : is a non-empty finite set of states
present in the finite control system.

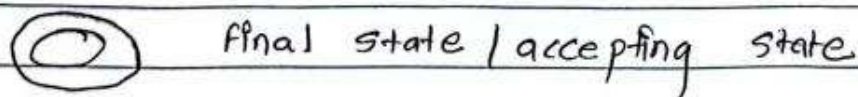
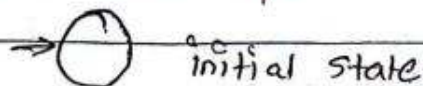
Σ : is an input alphabet

q_0 : is an initial state

F : is a set of final states $p \in Q$

δ : is a transition function defined from
 $Q \times \Sigma$ to Q

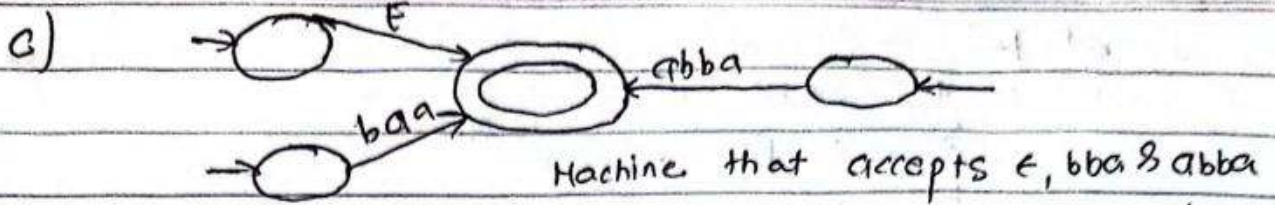
Transition Graph



Examples

a) Machine that accepts nothing

b) Machine that accepts all
string.



Design Example

Design a deterministic finite automata (DFA) is that accepts the language given by $L = \{w \in \{a,b\}^* : w \text{ has even number of } b\}$

Solⁿ let $M = (Q, \Sigma, \delta, s, F)$ be

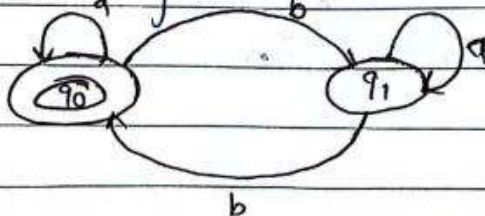
where,
 $Q = \{q_0, q_1\}$
 $\Sigma = \{a, b\}$
 $P = q_0$
 $F = \{q_0\}$

And the transition function δ is as follow:

Transition table

State \ Input	a	b
q_0		
q_1		

State Diagram



let us test for a string
 $(q_0, abab) \rightarrow (q_1)$

$(q_0, abab) \xrightarrow{a} (q_1, bab)$
 $\xrightarrow{b} (q_2, ab)$
 $\xrightarrow{a} (q_3, b)$ ~~Accepted~~
 $\xrightarrow{b} (q_0, \epsilon)$ Accepted !!

Design a deterministic finite automate (DFA) that accepts a language given by

$L = \{ w \in \{a, b\}^+ : w \text{ does not contain three consecutive b's} \}$

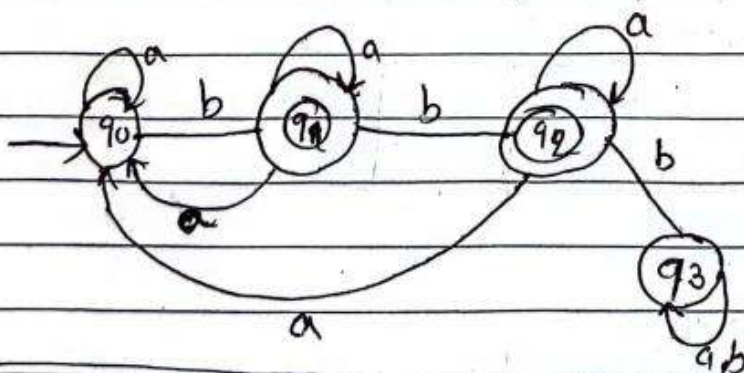
Soln let $M = (Q, \Sigma, \delta, s, F)$ be required DFA

where, $Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{a, b\}$

$s = q_0$

$F = \{q_0, q_1, q_2\}$



state	diagram	
q_0	q_0	q_1
q_1	q_0	q_2
q_2	q_0	q_3
q_3	q_3	q_3

3. Design a DFA that accepts a language given by $w = \{w \in \{0,1\}^*, w \text{ begins with } 01\}$
4. Design a DFA that accepts a language given by $L = \{w \in \{0,1\}^* : w \text{ begins with } 01 \text{ and has even length}\}$
5. Design a DFA that accepts a language given by $L = \{w \in \{0,1\}^* : w \text{ has neither } 00 \text{ nor } 11 \text{ as substrings}\}$
6. Design a DFA that accepts a language given by $L = \{w \in \{a,b\}^* : w \text{ has number of 'a' multiple of } 4\}$
7. Design DFA for the following language given are $\{0,1\}$
- a. All strings with even no of 0's and even no of 1's
 - b. All strings of length at most 4.

Solutions :-

- (3) DFA that accepts a language w begins with 01

Non-Deterministic Finite Automata (NFA OR NDFA)

NDFA has ability to change states in a way that is only partially determined by the current state and input symbol. There will be several possible "next states" for a given combination current state and input symbol.

NDFA needs the input string and choose at each step to go into any of the legal next states, the choice is not determined by anything in the model and is therefore said to be non-deterministic.

Formal Definition:

A non-deterministic automation (NDFA) is a quintuple.

$$M = (Q, \Sigma, \Delta, s, F)$$

Where,

Q : is a finite set of states

Σ : is a set of input symbol

$s = s \in Q$ is an initial state

$F = F \subseteq Q$ is a final state

Δ = is transition function mapping from $Q \times \Sigma \rightarrow 2^Q$ which is power set of Q .

Example 1: Design a non-deterministic finite Automata (NFA) that accepts the set of strings containing

an occurrence of the pattern bb or of the pattern bab.

Solⁿ

Let $M = (Q, \Sigma, \Delta, S, F)$ be the required NFA

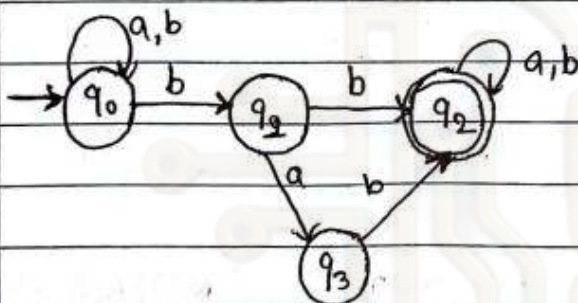
where, $Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{a, b\}$

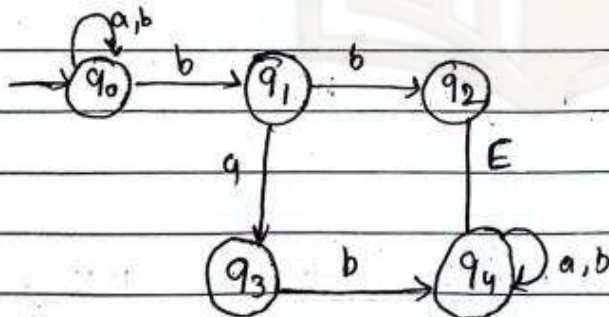
$S = q_0$

$F = \{q_2\}$

And transition is given by



$Q \backslash \Sigma$	a	b	ϵ
q_0	q_0	q_0, q_1	-
q_1	q_3	q_2	-
q_2	ϕ	ϕ	q_1
q_3	ϕ	q_1	-
q_4	q_4	q_4	-



where, $Q = \{q_0, q_1, q_2, q_3, q_4\}$

$\Sigma = \{a, b\}$

$S = q_0$

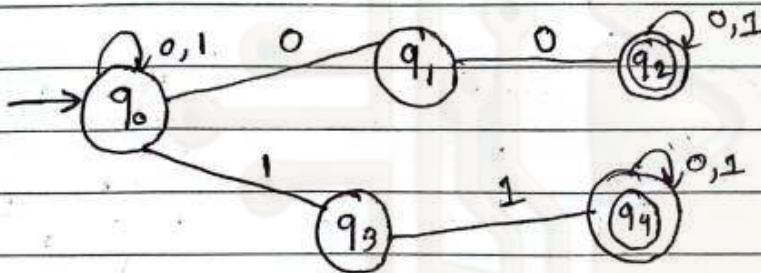
$F = \{q_2\}$

#2 Design a NFA for the language $L =$ all strings over $\{0,1\}$, that have at least two consecutive 0's or 1's.

Solⁿ

Let $M = (Q, \Sigma, \Delta, S, F)$ be the required NFA

where, $Q = \{q_0, q_1, q_2, q_3, q_4\}$
 $\Sigma = \{0, 1\}$
 $S = q_0$
 $F = \{q_2, q_4\}$

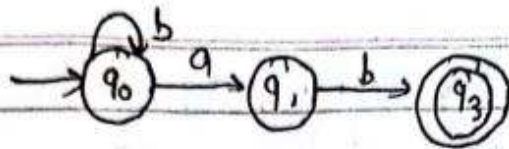


$Q \backslash \Sigma$	0	1
q_0	q_0, q_1	q_0, q_3
q_1	q_2	$\emptyset$
q_2	q_2	q_2
q_3	$\emptyset$	q_4
q_4	q_4	q_4

#3 NDFA for $R = b^*ab \mid bb^*ab$

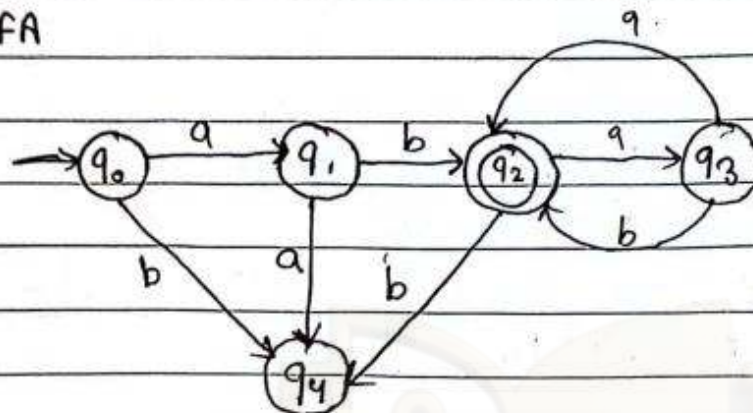
Solⁿ

Let $M = (Q, \Sigma, \Delta, S, F)$ be the required NDFA

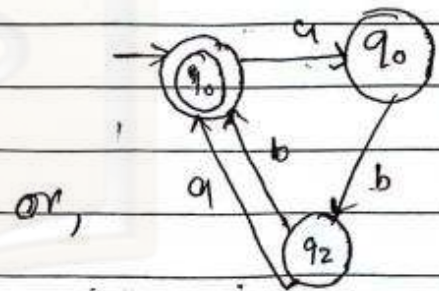
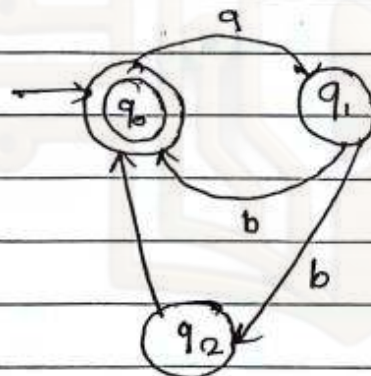


#4- Design a DFA for a regular expression $(ab \cup aba)^*$. And also design NFA for same regular expression.

DFA



NFA



NFA for $(ab \cup aba)^*$

NFA for $(ab \cup aba)^*$

#5 Find the NFA with four state for the language $L = \{a^n : n \geq 0\} \cup \{b^n a : n \geq 1\}$

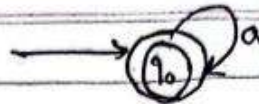
Soln

$$L = L_1 \cup L_2$$

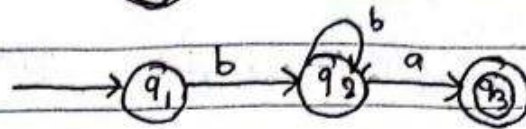
$$L_1 = a^n : n \geq 0$$

$$L_2 = b^n a : n \geq 1$$

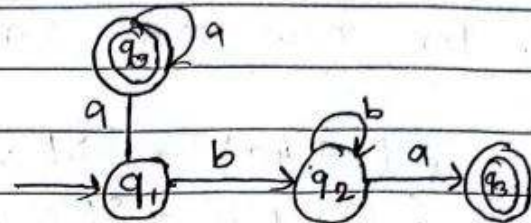
NFA for L_1 is



NFA for L_2 is



NFA for $L_1 \cup L_2$ is



Equivalence of DFA and NFA

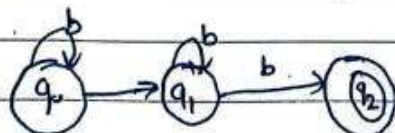
"For each non-deterministic finite automation there is an equivalent deterministic finite automation."

Steps:-

1. Observe all the transitions from starting state q_0 for every symbols in Σ .
2. For new states appeared in step 1, check all the transition for every Σ .
3. Repeat step 1, till new states are appeared.

Example:-

convert the following NFA into equivalent DFA



Solution:-

Step 1: Here initial state is q_0 , so we write (q_0) for new DFA

Then, $\delta(q_0, a) = \delta(q_0, a) = q_1$ new state.

$\delta(q_0, b) = \delta(q_0, b) = q_0$ old state.

step 2: For new state q_1 , we have

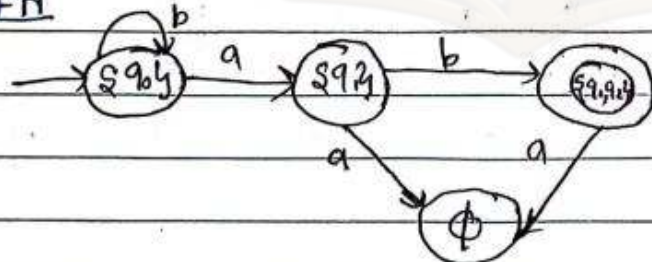
$$\delta(q_1, a) = \delta(q_1, a) = \phi$$

$$\delta(q_1, b) = \delta(q_1, b) = q_1, q_2 \text{ new state.}$$

step 3: For new state q_1, q_2 where,
 $\delta(q_1, q_2, a) = \delta(q_1, a) \cup \delta(q_2, a)$
 $= \phi \cup \phi = \phi$

$$\begin{aligned} \delta(q_1, q_2, b) &= \delta(q_1, b) \cup \delta(q_2, b) \\ &= q_1, q_2 \cup \phi \\ &= q_1, q_2 \text{ old states.} \end{aligned}$$

DFA



δ	a	b
q_0	q_1	q_0
q_1	ϕ	q_1, q_2
q_1, q_2	ϕ	q_1, q_2

Conversion of E-NFA to DFA

E-Transitions are the transition made without symbols

steps:

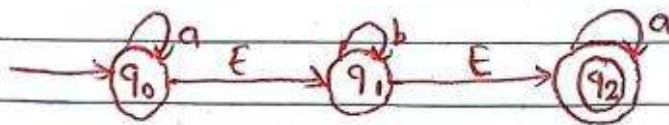
1. Find out the ϵ -closure of starting state and calculate the union of ϵ -closure and transitions of every symbol of Σ .

Note:- ϵ -closure is a set of the states in ENFA which can be reached without consuming symbols.

2. Repeat the same process till new states are appeared.

Example:

convert the following NFA to DFA



Soln

ϵ -closure of q_0 is ϵ -close (q_0) or

$$E(q_0) = \{q_0, q_1, q_2\} \quad \text{Similarly,}$$

$$E(q_1) = \{q_1, q_2\}$$

$$E(q_2) = \{q_2\}$$

Now,

$$\delta'(\{q_0, q_1, q_2\}, a) = \delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a)$$

$$= E(q_0 \cup \phi \cup q_2) \\ E(q_0) \cup E(q_2) \\ (q_0, q_1, q_2) \cup \{q_2\} \\ = \{q_0, q_1, q_2\}$$

$$\delta'(\{q_0, q_1, q_2\}, b) = E(\delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_2, b))$$

$$= E(\phi \cup q_1 \cup \phi) \\ = E(q_1) = \{q_1, q_2\} \text{ new state.}$$

Now, using new state,

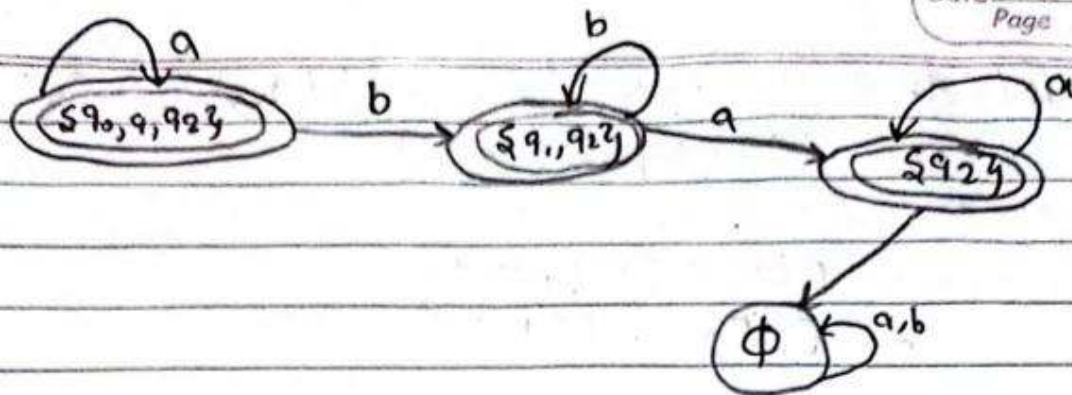
$$\delta'(\{q_1, q_2\}, a) = E(\delta(q_1, a) \cup \delta(q_2, a)) \\ = E(\phi \cup q_2) \\ = E(q_2) = \{q_2\} \text{ new state.}$$

$$\delta'(\{q_1, q_2\}, b) = E(\delta(q_1, b) \cup \delta(q_2, b)) \\ = E(q_1 \cup \phi) = E(q_1) = \{q_1, q_2\} \text{ old state.}$$

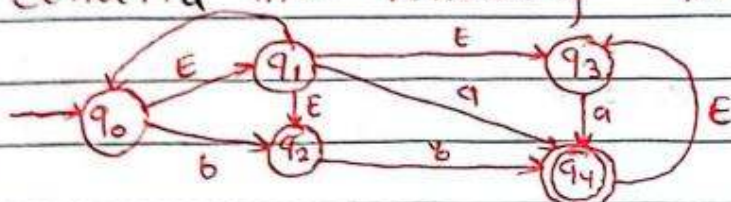
using new state $\{q_2\}$

$$\delta'(\{q_2\}, a) = E(q_2, a) = \{q_2\} \\ = E(\delta(q_2, a)) = E(q_2) = \{q_2\} \text{ old state}$$

$$\delta'(\{q_2\}, b) = E(\delta(q_2, b)) = E(\phi) = \phi$$



convert the following NFA to DFA



ϵ -closure of q_0 is ϵ -close (q_0) or

$$E(q_0) = \{q_0, q_1, q_2, q_3\}$$

$$E(q_1) = \{q_3, q_2, q_4\}$$

$$E(q_2) = \{q_2\}$$

$$E(q_3) = \{q_3\}$$

$$E(q_4) = \{q_3, q_4\}$$

$$\text{Now, } \delta'(\{q_0, q_1, q_2, q_3\}, a) = \epsilon(\delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a) \cup \delta(q_3, a))$$

$$= E(\phi \cup \phi)$$

$$E(\phi \cup q_0 \cup \phi \cup q_4)$$

$$E(q_0) \cup E(q_4)$$

$$(\{q_1, q_2, q_3\} \cup \{q_3\})$$

$$= \{q_1, q_2, q_3\}$$

$$\{q_0, q_1, q_2, q_3\} \cup \{q_3, q_4\}$$

$$= \{q_0, q_1, q_2, q_3, q_4\} \text{ new state}$$

δ' Now using

$$\delta'(\{q_0, q_1, q_2, q_3\}, b) = E(\delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_2, b), \delta(q_3, b))$$

$$= E(q_2 \cup \phi \cup q_4 \cup \phi)$$

$$= E(q_2) \cup E(q_4)$$

$$= (q_2) \cup (q_3, q_4)$$

$$= (q_2, q_3, q_4) \text{ new state.}$$

Now, using new state,

$$\delta'(\{q_0, q_1, q_2, q_3, q_4\}, a) = E(\delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a) \cup \delta(q_3, a) \cup \delta(q_4, a))$$

$$E(\phi \cup q_4 \cup \phi \cup q_4 \cup \phi)$$

$$E(q_4) \cup E(q_4)$$

$$= (q_3, q_4) \text{ new state}$$

$$\delta'(\{q_0, q_1, q_2, q_3, q_4\}, b) = E(\delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_2, b) \cup \delta(q_3, b) \cup \delta(q_4, b))$$

$$= E(q_2 \cup \phi \cup q_4 \cup \phi)$$

$$= E(q_2) \cup E(q_4)$$

$$= (q_2, q_3, q_4) \text{ new state.}$$

$$\delta'(q_2, q_3, q_4), a) = E(\delta(q_2, a), \delta(q_3, a), \delta(q_4, a))$$

$$= E(\phi \cup q_4 \cup \phi)$$

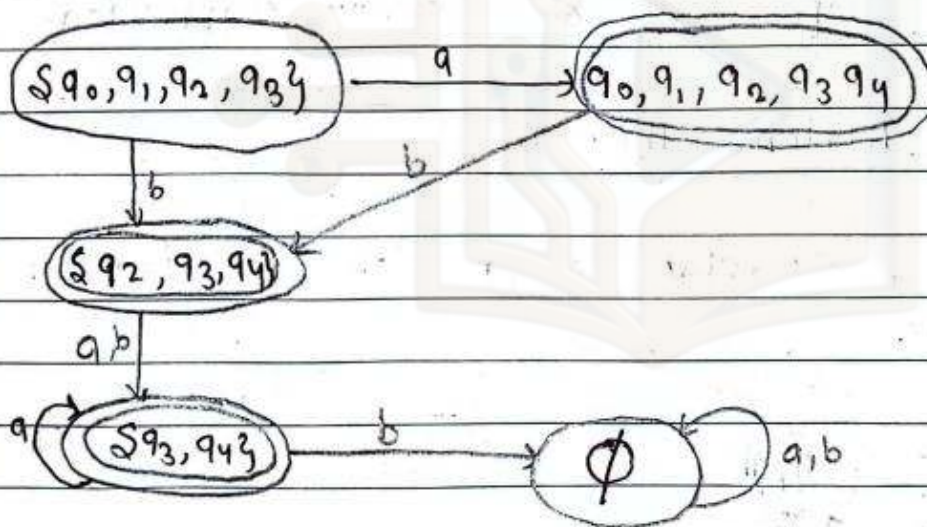
$$= E(q_4)$$

$$= (q_3, q_4) \text{ new state.}$$

$$\begin{aligned} \delta'(q_2, q_3, q_4, b) &= E(\delta(q_2, b), \delta(q_3, b), \delta(q_4, b)) \\ &= E(q \cup \phi \cup \phi) \\ &= E(q) \\ &= (q_3, q_4) \quad \text{new state.} \end{aligned}$$

$$\begin{aligned} \delta'(\{q_3, q_4\}, a) &= E(\delta(q_3, a), \delta(q_4, a)) \\ &= E(q_4 \cup \phi) \\ &= (q_3, q_4) \quad \text{old state.} \end{aligned}$$

$$\begin{aligned} \delta'(q_3, q_4, b) &= E(\delta(q_3, b), \delta(q_4, b)) \\ &= E(\phi \cup \phi) \\ &= E(\phi) \\ &= \phi \end{aligned}$$



Arden's Theorem

Let P and Q be two regular expressions over alphabet Σ . If P does not contain null string ϵ , then

$$R = Q + RP$$

has unique solution that is $R = QP^*$

This can be understood as $R = \emptyset + RP$

Let's put the value of R in RHS.

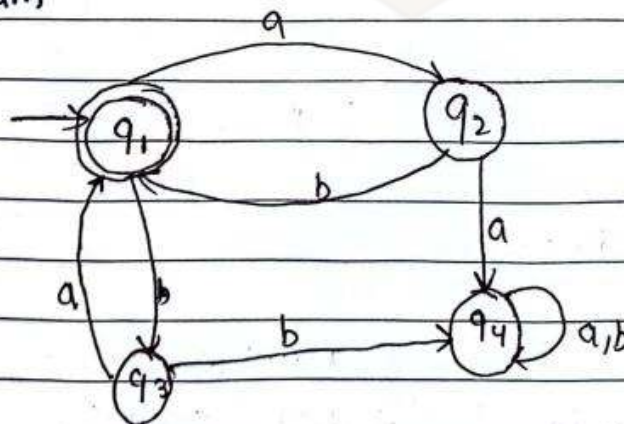
$$R = \emptyset + (\emptyset + RP)P = \emptyset + \emptyset P + RP^2$$

When we put the value of R again, we get the following equation.

$$\begin{aligned} R &= \emptyset + \emptyset P + \emptyset P^2 + \emptyset P^3 + \dots \\ &= \emptyset (1 + P + P^2 + P^3 + \dots) \\ &= \emptyset (E + P + P^2 + P^3 + \dots) \\ &= \emptyset P^* \text{ (By the definition of closure operation of regular expression.)} \end{aligned}$$

Use of Arden's theorem:-

#1) Find the regular expression for transition diagram



Starting state
rakhni

* q4 dead
unajore

Now, let's find the equations:

$$q_1 = E + q_2 b + q_3 a$$

$$q_2 = q_1 a$$

$$q_3 = q_1 b$$

$$q_4 = q_2 b + q_4 a + q_4 b + q_2 a$$

put the value of q_2 and q_3 in q_1 as,

$$q_1 = E + q_2 ab + q_3 ba$$

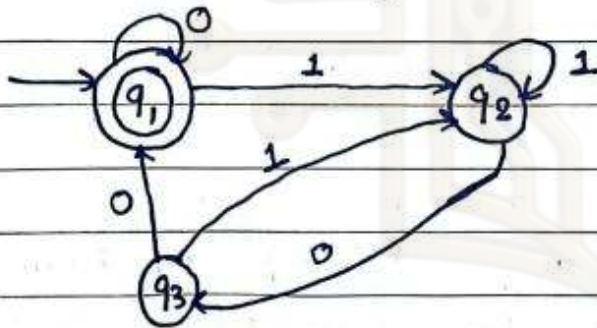
$$= E + q_1 (ab + ba)$$

comparing with Arden's theorem, $R = \emptyset + RP$
 $\Rightarrow R = \emptyset P^*$

$$q_1 = E(ab + ba)^*$$

So, the required regular expression is $(ab + ba)^*$

Construct a regular expression corresponding to the state diagram.



Now, let's find the equations!

$$q_1 = E + q_1 0 + q_3 0$$

$$q_2 = q_2 1 + q_3 1 + q_1 1$$

$$q_3 = q_2 0$$

q

$$q_2 = q_1 1 + q_2 1 + (q_2 0) 1 = q_1 1 + q_2 (1 + 01)$$

$$q_2 = q_1 1 (1 + 01)^*$$

q1

$$\text{So, } q_1 = q_1 0 + q_3 0 + E = q_1 0 + q_2 00 + E = q_1 0 + (q_1 1 (1 + 01)^*) 00 + E$$

$$q_1 = q_1 (0+1 (1+01)^* 00) + \epsilon$$

$$q_1 = \epsilon (0+1 (1+01)^* 00)^*$$

So regular expression is $(0+1 (1+01)^* 00)^*$

Regular language

Regular languages can be defined from the empty set and from some finite number of singleton sets, by the operations of union, composition (concatenation) and Kleen closure. Specially, consider any alphabet Σ , then a regular sets over Σ is defined in the following way.

The empty set ϕ , the set $\{ \epsilon \}$ is defined to the following

- (a) The empty set ϕ , the set $\{ \epsilon \}$ containing empty string and the set $\{ a \}$ for each symbol a in Σ are regular sets.
- (b) If L_1 and L_2 are regular sets, then so are the union $L_1 \cup L_2$, the composition $L_1 L_2$ and the Kleen closure L_1^*
- (c) No other is regular

Theorem:

A set is a regular set if and only if it is accepted by a finite state automation.

→ Regular sets of the form $\emptyset, \{a\}, L_a, L_b, L_a^*$ are quite often denoted by $\emptyset, \epsilon, a, (\alpha) + (B)$ ~~and~~ $(\alpha)(B)$ and $(\alpha)^*$ respectively where α and B are assumed to be the expressions that denote ~~the~~ L_a and L_b in a similar manner respectively and a is assumed to be a symbol from the alphabet. Expressions that denote regular set are regular expression

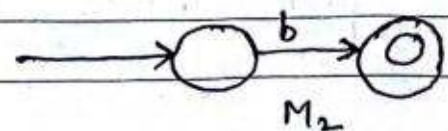
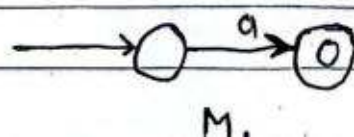
It is evident that the empty set and all singulation are indeed accepted by the finite automata. The finite automata languages are closed under union, concatenation, and Kleen star. hence, every regular set (language) is accepted by some finite automation.

Construction of NFA from regular Expression

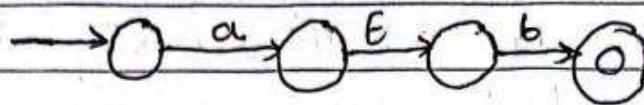
Given R.E $\rightarrow (ab \cup ba)^*$

We can construct required NFA in following steps.

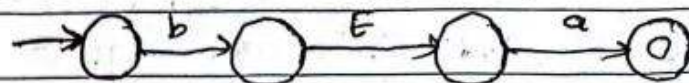
Step 1: a, b



step 2: $ab:ba$

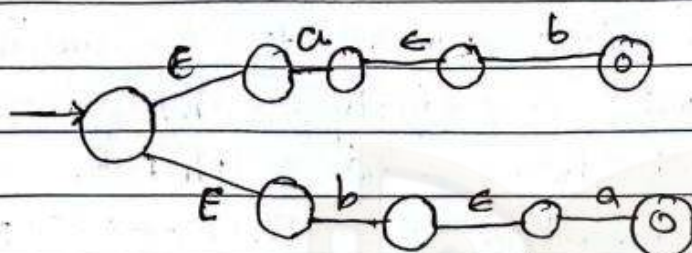


$M_1 = ab$



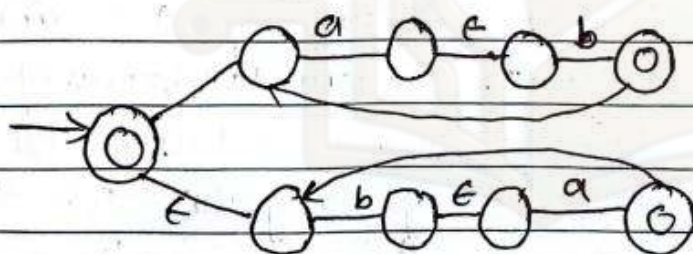
$M_2 = ba$

step 3



$M = ab \cup ba$

step 4

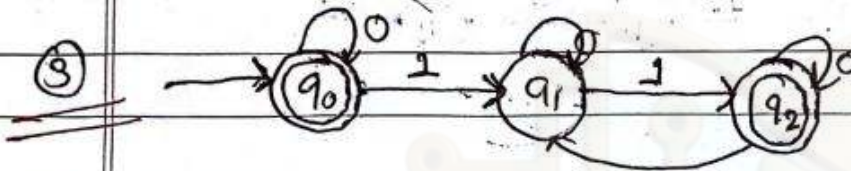
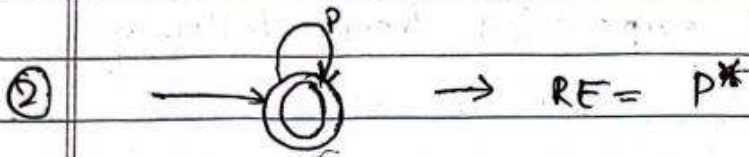
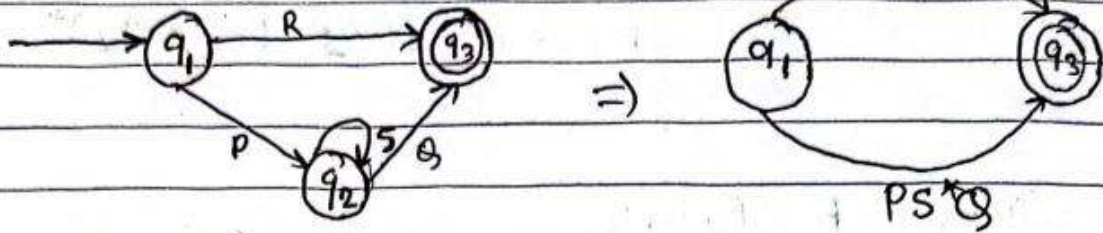


$M = (ab \cup ba)^*$

Equivalence of finite automata and Regular Expression

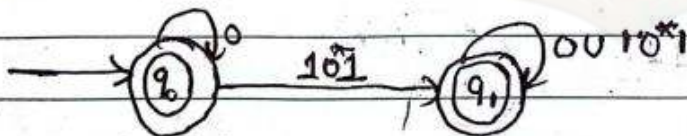
We can convert each of the regular expression into corresponding finite automata and can also find out the regular expressions from given finite automata as follow.

Example 1:

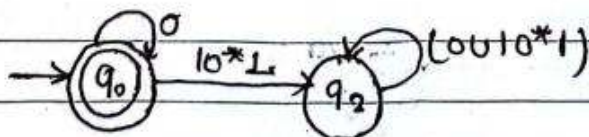


Construction of NFA from regular expression.
Solⁿ

here, q_1 is neither Initial state nor final state, so we can eliminate q_1 as follows:

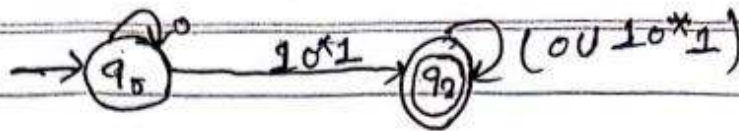


Now, turn off state q_2 , we get



here, q_2 acts as dead state, so Regular expression $R_1 = 0^*$

Again Turn off q_0 , we get,

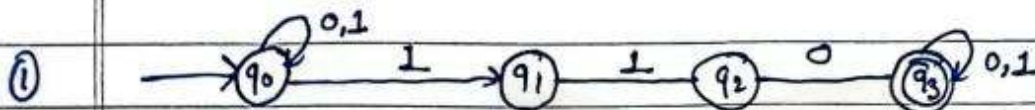


$$R_2 = 0^* 10^* 1 (0U10^* 1)^*$$

Then, $R = R_1 U R_2$
 $= 0^* U 0^* 10^* 1 (0U10^* 1)^*$

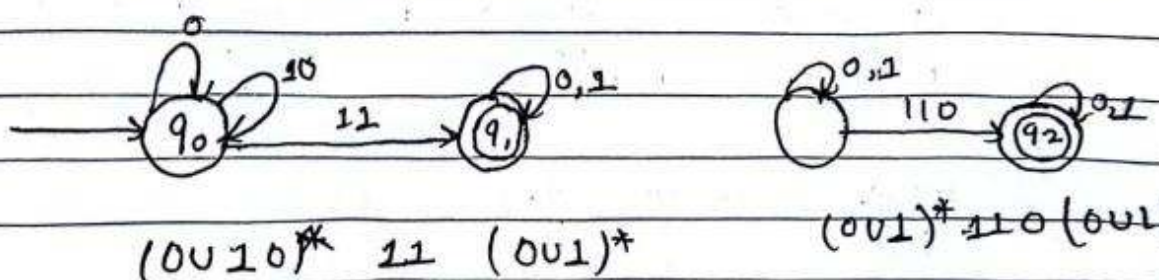
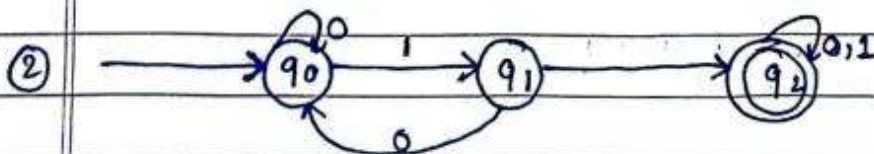
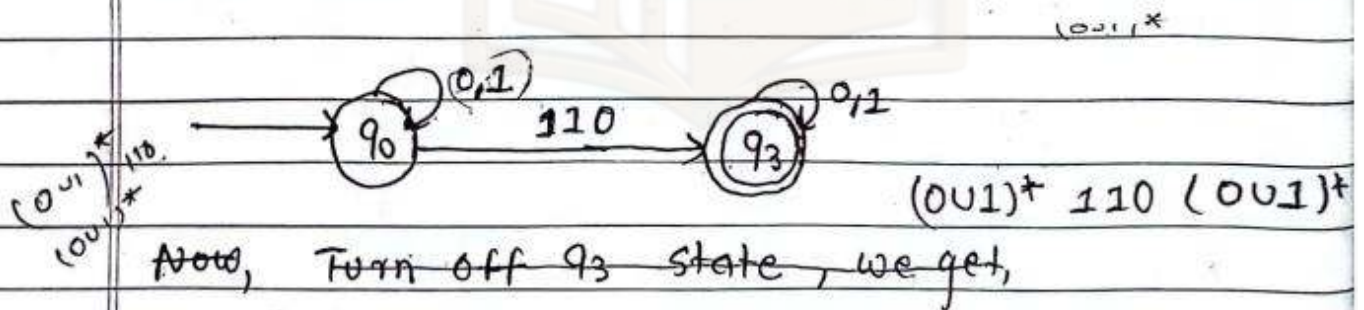
Q10

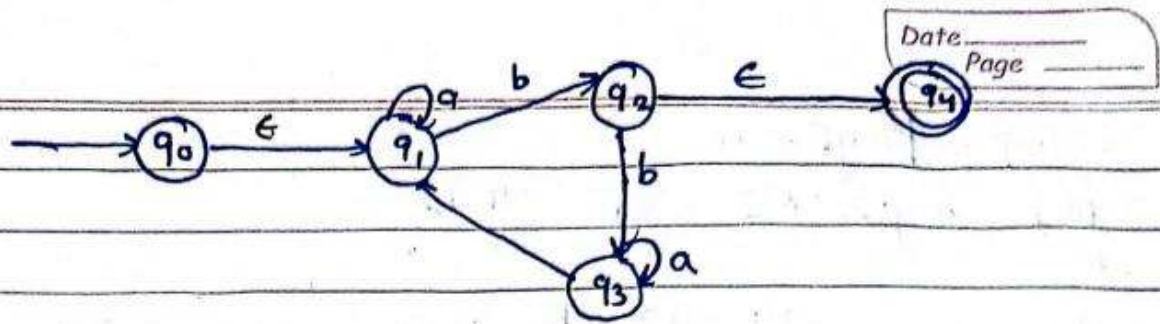
Find the regular expression from following NFA



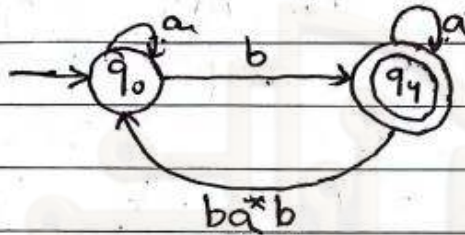
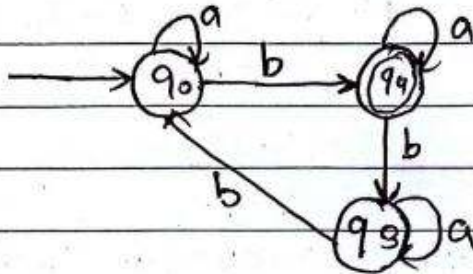
→ $0U1^n$

here, q_1 and q_2 neither initial state nor final state, so we can eliminate q_1 & q_2 as follows:





solⁿ removing ϵ transition



a^*

$$\Rightarrow a^* b a^* (ba^* ba^* b)^*$$

Pumping lemma for regular set (language)

statement

Let L be a regular language. And w be any string $w \in L$, n be a pumping constant with $n \geq 1$ ϕ $1001 \geq n$.

Then we can break w into three substrings $w = xyz$ such that

i) $y \neq \epsilon$ or $y \neq \phi$

- (ii) $|ny| \leq n$
 (iii) $xy^iz \in L$ for all $i \geq 0$

i.e, we always find a non-empty string y not too far from the beginning of w that can be pumped. It means separating y any number of times keeps the resulting string in the language.

Proof:

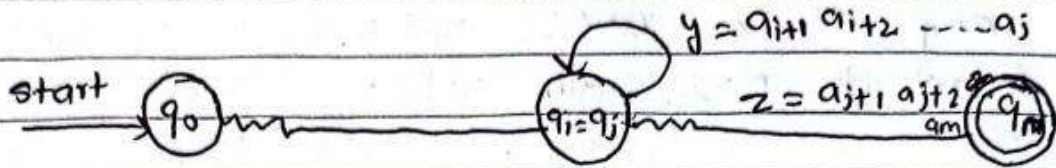
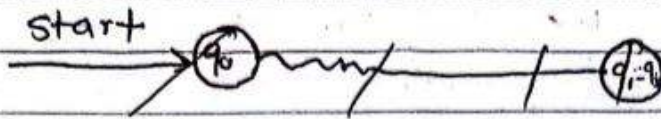
Suppose L is a regular language. Then $L = L(M)$ for some DFA, M . Suppose M has n states. Now, consider any string w of length n or more. Say $w = a_1 a_2 a_3 \dots a_n$, where $m \geq n$ and each a_i is an input symbol, for $i = 0, 1, 2, \dots, m$.

Let $\delta(q_0, q_1, q_2, \dots, q_i) = q_r$, But it is not possible for each of the $n+1$ states $q_0, q_1, q_2, \dots, q_n$ to be distinct, since there are only n different states. Thus we can find two different integers i and j each $0 \leq i < j \leq n$ such that $q_r = q_i$. Now we can break $w = xyz$ as follows:

$$x = a_1 a_2 \dots a_i$$

$$y = a_{i+1} a_{i+2} \dots a_j$$

$$z = a_{j+1} a_{j+2} \dots a_m$$



Note :- that x may be empty in case that $i=0$, also z be empty if $j = n-1$ but y can be empty since $i < j$.

Now, when the automaton M receives $xyiz$ for any $i \geq 0$, if $i=0$, then the automaton, M with i/p x on start state q_0 reaches state $q_i (= q_0)$. Then for i/p z at start $q_i = q_0$, it goes from q_i to the accepting state q_n . Thus, it accepts xz .

If $i > 0$, then m goes from q_0 to q_1 on i/p x , circles from q_1 to $q_i (= q_j)$ i times on i/p y , and then goes to the accepting state on i/p z .

Thus for any $i \geq 0$, $xyiz$ is also accepted by M that is $xyiz$ is in L .

Pumping lemma for Regular language

Proof! let L be a regular language w be any string $w \in L$ and n be a pumping constant $n \geq 1$ with $|w| \geq n$

Then w can be re-written as $w = xyz$ such that

$$|xy| \leq n$$

$$|y| \geq 1$$

$$xy^iz \in L \text{ for all } i \geq 0$$

From pumping lemma, we can write

$$w = xyz = a^n b^n$$

which can be written as

$$w = xyz = \overbrace{a \dots a}^n \overbrace{a \dots a}^n \overbrace{ab \dots ab}^n$$

$\underbrace{\hspace{1.5cm}}_x \quad \underbrace{\hspace{1.5cm}}_y \quad \underbrace{\hspace{1.5cm}}_z$

here, $y = a^k, k \geq 1$

using pumping lemma

for $i \neq 0$, we get

$$x_2 = a^{n+k} b^n \notin L$$

which contradicts the theorem, hence the given language

$$\{a^n b^n \mid n \geq 1\} \text{ is not regular.}$$

Proved #

Example 2: Prove that the language
 $L = \{a^m b^{2m} : m \geq 1\}$ is not regular language

Proof! Let L be a regular language w be any
 $w \in L$ and n be a pumping constant $m \geq 1$
 with $|w| \geq m$

Then w can be re-written as $w = xyz$ such
 that

$$|xy| \leq m$$

$$|y| \geq 1$$

$$xy^iz \in L \text{ for all } i \geq 0$$

for pumping lemma, we can write

$$w = xyz = a^m b^{2m}$$

which can be written as

$$w = xyz = \overbrace{aa \dots aa}^m \overbrace{ab \dots bb}^{2m}$$

$\underbrace{\hspace{1.5cm}}_x \quad \underbrace{\hspace{1.5cm}}_y \quad \underbrace{\hspace{1.5cm}}_z$

here,

$$y = a^k, k \geq 1$$

using pumping lemma

for $i=0$, we get

$$xz = a^{m+k} b^{2m} \notin L$$

which contradicts the theorem, hence the
 given language,

$$L = \{a^m b^{2m} : m \geq 1\} \text{ is not regular}$$

#3- show that $L = \{a^m! : m \geq 1\}$ is not regular by using pumping lemma for regular language.

Proof:

Let L be a regular language w be any string $w \in L$ and n be a pumping constant

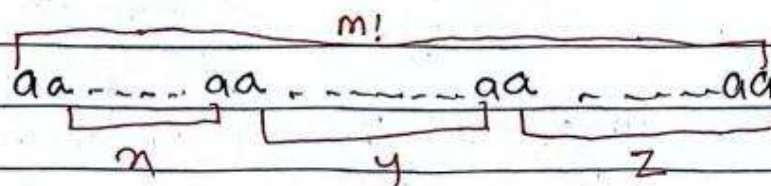
$$w = xyz = a^m$$

$$\text{If } y = a^k, k \geq 1$$

using pumping lemma, $i \geq 2$

$$xy^iz = xy^2z = a^{m+k}$$

If $m+k \neq p!$ (i.e. perfect factorial)



here

If $y = a^k$, $k \geq 1$
Using pumping lemma
for $i=2$

$$xy^2z = xyzy = \underline{a^{m!+k}}$$

If $m! + k = p!$ (i.e. perfect factorial).

$$m! + k \leq m! + m$$

$$< m! + m! \text{ for } m > 1$$

$$< m!m + m!$$

$$= m!(m+1)$$

$$= (m+1)!$$

$m! + k \neq (m+1)!$ $(m+1)! \geq p!$ hence,
 $a^{m!+k} \in L$

Closure properties of languages that are
accepted by finite automata.

Chapter - 3

Date _____
Page _____

Content free grammar (CFG)

Introduction:-

Let us take regular expression $a(aub)^*b$. This expression gives strings consisting of a leading a followed by mid part $(aub)^*$ followed by trailing b .

If S is a new symbol then we can write

$$S \rightarrow aMb$$

$$M \rightarrow aM$$

$$M \rightarrow bM$$

$$M \rightarrow \epsilon$$

A content free grammar is a language generator that operates on the rules like above.

Now, let us take generation of aab for this, we have to replace M by aM , whatever be the particular or ~~surroundings~~ surroundings, so, it is called content free.

This can be obtained as,

$$\begin{aligned} S &\Rightarrow aMb & [\because S &\rightarrow aMb] \\ &\Rightarrow aaMb & [\because S &\rightarrow aM] \\ &\Rightarrow aab & [\because S &\rightarrow \epsilon] \end{aligned}$$

Formal Definition:-

A context free grammar is a finite set of variables (called non-terminals or syntactic categories) each of which represent a language. The languages represented by variables are described recursively in terms of each other and primitive symbols called terminals. The rules relating the variables are called productions. Mathematically,

CFG is defined as quadruple

$$G = (V_n, V_t, P, S)$$

where, V_n = finite set of non-terminals, generally represented by capital letters A, B, C, D

V_t = finite set of terminals, generally represented by small letters a, b, c, ...

P = set of rules or productions of CFG

S = starting non-terminals, called start symbol of the grammar $S \in V_n$

G is context free and all productions in P have the form $\alpha \rightarrow \beta$

where, $\alpha \in V_n$ and $\beta \in (V_t \cup V_n)^*$

some Terminologies

Derivation:

Let $G = (V_n, V_t, P, S)$ be a context free grammar (CFG). If $w_1, w_2, w_3, \dots, w_n$ are strings over $(V_n \cup V_t)^*$ such that

$w_1 \Rightarrow w_2 \Rightarrow w_3 \dots \Rightarrow w_n$ then we can say w_n is derivable from w_1 . w_1 is called derivation.

language of context free grammar $L(G)$:

If $G = (V_n, V_t, P, S)$ is a CFG, then the language of G denoted by $L(G)$ is the set of terminal strings that have derivations from the start symbol.

$$\text{i.e. } L(G) = \{w \in V_t^* : S \xRightarrow{*} w\}$$

Sentential Form

Derivations from the start symbol produce strings that have a special role. This is called sentential form. That is, if

$G = (V_n, V_t, P, S)$ is a CFG, then any string α in $(V_n \cup V_t)^*$ such that $S \xRightarrow{*} \alpha$ is a sentential form.

Example:-

1. Consider a grammar $G = (V_n, V_t, P, S)$ where,

$V_n = \{S\}$, $V_t = \{a, b\}$ and set of production P is given by

$$P = \{ S \rightarrow aSb, \\ S \rightarrow ab \}$$

here S is the only non-terminal which is the starting symbol of the grammar. 'a' and 'b' are the terminals and there are two productions $S \rightarrow aSb$ and $S \rightarrow ab$.

Now, we will ~~show~~ show that how the string a^2b^2 can be derived we have,

$$\begin{aligned} S &\Rightarrow aSb \\ &\Rightarrow aabb \quad [\because S \rightarrow ab] \\ &\Rightarrow a^2b^2 \end{aligned}$$

here, we need to apply first production and then the second one. By applying first production $(n-1)$ times, followed by second production, we get,

$$\begin{aligned} S &\Rightarrow aSb \\ &\Rightarrow aaSbb \\ &\Rightarrow aaaSbbb \\ &\vdots \\ &\Rightarrow a^{n-1}Sb^{n-1} \\ &\Rightarrow a^{n-1}abb^{n-1} \\ &\Rightarrow a^nb^n \end{aligned}$$

Thus the language for the above grammar is
 $L(G) = \{a^nb^n : n \geq 1\}$

#(2) let $G = (V_n, V_t, P, S)$ where, $V_n = \{S\}$, $V_t = \{a, b\}$,
 $P = \{S \rightarrow aas, S \rightarrow b\}$

Now, find $L(G)$

solⁿ

$S \Rightarrow aas$

$\Rightarrow aaas$

$aa \quad aa \quad aas$

$(aa)^n S$

$(aa)^n b$

hence the language of given grammar
 G is

$$L(G) = \{(aa)^n b : n \geq 0\}$$

#3) let $G = (V_n, V_t, P, S)$, $V_n = \{S, A\}$, $V_t = \{a, b\}$

$P = \{ S \rightarrow aA,$
 $A \rightarrow bs,$
 $S \rightarrow \epsilon \}$

$S \rightarrow aA$

$\Rightarrow abs$

$\Rightarrow abaA$

$\Rightarrow ababs$

$\Rightarrow (ab)^n S$

$\Rightarrow (ab)^n //$

hence, the language of the given grammar is
 $L(G) = \{(ab)^n : n \geq 0\}$

Writing content free Grammar examples :-

1. Write a content free Grammar (CFG) for the language given by $L = \{a^n b^n; n \geq 0\}$

soln

let $G = (V_n, V_t, P, S)$ be required CFG where

$$V_n = \{S\}$$

$$V_t = \{a, b\}$$

$$P = \{S \rightarrow asb, S \rightarrow \epsilon\}$$

lets test for $aaabbbb$

$$S \Rightarrow asb$$

$$\Rightarrow aasbb$$

$$\Rightarrow aaasbbb$$

$$\Rightarrow aaasbbb \quad [\because S \rightarrow \epsilon]$$

$$\Rightarrow a^4 b^4 \text{ produced } \#$$

2. Write CFG for the language $L = \{a^m b^n; m \geq n\}$

soln

let $G = (V_n, V_t, P, S)$ be the required CFG
when,

$$V_n = \{S\}, V_t = \{a, b\}$$

$$P = \{S \rightarrow as,$$

$$S \rightarrow asb,$$

$$S \rightarrow \epsilon\}$$

3. For equal number of a ' and b '.

$$P = \{ S \rightarrow asb, \\ S \rightarrow bsa, \\ S \rightarrow SS, \\ S \rightarrow \epsilon \} \#$$

4. write a context free grammar (CFG) for $w = w^R$

$$\Rightarrow \begin{aligned} S &\rightarrow asa \\ S &\rightarrow bsb, \\ S &\rightarrow \epsilon, \\ S &\rightarrow a, \\ S &\rightarrow b \end{aligned}$$

5. $w = w^R$

$$P = \{ S \rightarrow asa, \\ S \rightarrow bsb, \\ S \rightarrow \epsilon \}$$

6. w has even length

$$P = \{ S \rightarrow asa, \\ S \rightarrow bsb, \\ S \rightarrow asb, \\ S \rightarrow bsa, \\ S \rightarrow \epsilon \}$$

7. w has odd length

$$P = \{ S \rightarrow asa, \\ S \rightarrow bsb, \\ asb \\ bsa \}$$

Derivation Table

A derivative tree or parse tree is an ordered tree in which nodes are labelled with the left side of phenomena production and in which the children of a node represent its corresponding right sides. It is tree representation of derivation.

Derivation:

Let $G = (V_n, V_t, P, S)$ be a cfG, Then a tree is a derivation tree is a derivation tree for G if,

- (i) Every vertex has a label, which is a symbol of $V_n \cup V_t \cup \{ \epsilon \}$
- (ii) The label of root is (i.e. starting non-terminal,
- (iii) If a vertex is interior and has label A, then A must be in V_n .
- (iv) If n has label A and vertices $n_1, n_2, n_3 \dots n_k$ are the children of vertex n, involve from the left, with labels $\gamma_1, \gamma_2, \dots, \gamma_k$ must be a production in P.
- (v) If vertex n has ϵ , then n is a leaf and is only son of its father (i.e. only child of the parent)

Yield of parse tree

If we look at the leaves of any parse tree and concatenate them from the left we get a string, which is called "the yield of the tree".

Eg:- Given $G = (V_n, V_t, P, s) = (\{s, A\}, \{a, b\}, P, s)$
 where P is defined as

$$S \rightarrow aAs \mid aSs,$$

$$A \rightarrow sba \mid ba$$

The derive the string as aabaa.

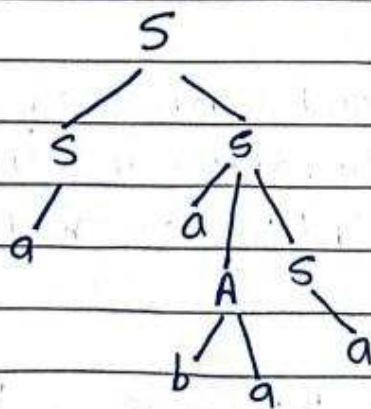
here, $S \rightarrow ss$

$$\Rightarrow as \quad [\because S \rightarrow a]$$

$$\Rightarrow aAs \quad [\because S \rightarrow aAs]$$

$$\Rightarrow aabas \quad [\because A \rightarrow ba]$$

$$\Rightarrow aabaa \quad [\because s \rightarrow a]$$



$\therefore$ yield is aabaa #

A2. Given $G = (V_n, V_t, P, S)$ where P is defined as

$$S \rightarrow aAs \mid a;$$

$$A \rightarrow sbA \mid ss \mid ba$$

Show that $s \xRightarrow{*} aabbaa$ & construct a derivation tree where yield is $aabbaa$

here, $S \rightarrow aAs$

$$\Rightarrow aAa$$

$$\Rightarrow asbA$$

$$\Rightarrow aabss$$

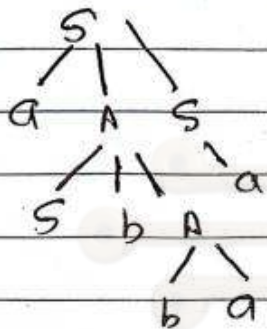
$$aabaa$$

$$[S \rightarrow a]$$

$$[A \rightarrow sbA]$$

$$[A \rightarrow ss]$$

$$[S \rightarrow a]$$



$$S \rightarrow aAs$$

$$\Rightarrow asbAS \quad [A \rightarrow sbA]$$

$$\Rightarrow aabAS \quad [S \rightarrow a]$$

$$\Rightarrow aabba \quad [A \rightarrow ba]$$

$$\Rightarrow aabbaa \quad [S \rightarrow a]$$

yield is $aabbaa$ //

Ambiguity of Grammar and language.

A context free grammar is called ambiguous if for at least one word in the language that it generates, there are two possible derivations of the word that can correspond to different syntax trees. If a CFG is not ambiguous, it is unambiguous.

Consider a CFG with production as

$S \rightarrow asb \mid ss \mid \epsilon$, prove that the given grammar is ambiguous for $aabb$.

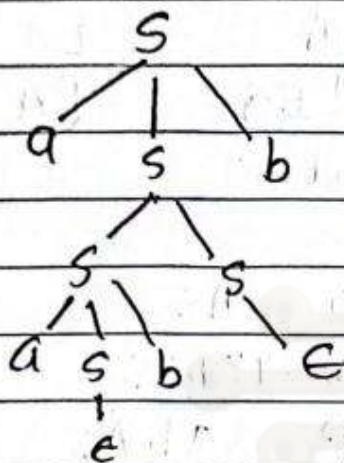
Soln

$$S \Rightarrow asb$$

$$\Rightarrow aasb \quad assb \quad [\because S \rightarrow ss]$$

$$\Rightarrow aasbsb \quad [\because S \rightarrow asb]$$

$$\Rightarrow aabb \quad [\because S \rightarrow \epsilon]$$



yield = aabb

Simplification of CFL

We get simplified grammar by applying -

(1) We must eliminate useless / non-generating / unreachable symbols from given grammar.

(2) We must eliminate empty-productions.

(3) We must eliminate unit productions.

(1) Elimination of useless symbols:-

→ We must eliminate useless. Say A is generating if $A \Rightarrow^* w$ for some terminal string w .

→ We say A is reachable if there is a derivation $S \Rightarrow^* \alpha A \beta$ for some α and β

Eg:- 1: Identify and eliminate useless symbols from following grammar.

$S \rightarrow aB | bX$

$A \rightarrow BA d | bS X | a$

$B \rightarrow aS B | bB X$

$X \rightarrow S B D | aB X | a d$

Here, A and X are generating due to $A \rightarrow a$ and $X \rightarrow a d$, S is also useful due to production $S \rightarrow bX$

$A \rightarrow bS X | a$

$X \rightarrow a d$

Again, A is unreachable symbol, so, we must eliminate this symbol to get,

$$S \rightarrow bX$$

$$X \rightarrow ad$$

This is simplified grammar

Eg:-2

$$S \rightarrow AB \mid CA$$

$$B \rightarrow Bc \mid AB$$

$$A \rightarrow a$$

$$C \rightarrow AB \mid b$$

2. Elimination of empty production (i.e ϵ -production)

Eg.1 Eliminate ϵ -production from following grammar

$$S \rightarrow AB$$

$$A \rightarrow AaA \mid \epsilon$$

$$B \rightarrow b$$

Soⁿ, here $A \rightarrow \epsilon$ is an empty production. We can eliminate this production as follows

$$\begin{aligned} S &\rightarrow AB \mid B \\ A &\rightarrow AaA \mid aA \mid Aa \mid a \\ B &\rightarrow b \end{aligned}$$

3. Elimination of unit production

Eg. 1:- Eliminate unit productions from following grammar.

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow a \\ B &\rightarrow C \mid d \\ C &\rightarrow D \\ D &\rightarrow E \\ E &\rightarrow a \end{aligned}$$

here, $B \rightarrow C$, $C \rightarrow D$ and $D \rightarrow E$ are unit productions. We can eliminate these as follows:-

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow a \\ B &\rightarrow a \mid d \\ C &\rightarrow a \\ D &\rightarrow a \\ E &\rightarrow a \end{aligned}$$

Again, C , D and E are unreachable symbols. So, we have to eliminate these symbols. Finally, we get

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow a \\ B &\rightarrow a \mid d \end{aligned}$$

Eg. 2 Eliminate unit production from following grammar.

$S \rightarrow aA|B$

$A \rightarrow bA|c$

$B \rightarrow abA|bc$

here, $S \rightarrow B$ in unit production,

$S \rightarrow aA|abA|bc$

$A \rightarrow bA|c$

$B \rightarrow abA|bc$

Chomsky Normal Form (CNF)

If a CFG has only production of the form

(i) $A \rightarrow BC$ where A, B , and C are each non-terminals

i.e non-terminal $\rightarrow$ exactly two non-terminals

(ii) $A \rightarrow a$ where ' A ' is a non-terminals

i.e non-terminal $\rightarrow$ exactly one terminal
is said to be chomsky normal form or CNF.

conversion to CNF

consider the grammar $G = (\{S, A, B\}, \{a, b\}, P, S)$
where S is the start symbol and the
production P is given by

$$S \rightarrow bA / aB$$

$$A \rightarrow bAA / as / a$$

$$B \rightarrow aBB / bS / b$$

find the equivalent CNF of it.

soln

Since a CNF has either two non-terminals
or one terminal at the right side, we have
to replace the terminal ' b ' by a non-terminal
 C_b and ' a ' by a_a in the first production. Then
we get,

3. Design a pushdown automata (PDA) M that accepts a language $L = \{w \in (a,b)^* \mid \text{number of } a \text{ is equal to number of } b\}$.

$$\begin{aligned} S &\rightarrow c_b A \mid c_a B \\ A &\rightarrow c_b AA \mid c_a S \mid a \\ B &\rightarrow c_a BB \mid c_b S \mid b \\ c_a &\rightarrow a, \quad c_b \rightarrow b \end{aligned}$$

Now in the second & third rule $c_b AA$ and $c_a BB$ are not in the CNF, as they contain 3 non-terminals. Replace AA by D and BB by E ,

we get,

$$\begin{aligned} S &\rightarrow c_b A \mid c_b B \\ A &\rightarrow c_b D \mid c_b S \mid a \\ B &\rightarrow c_a E \mid c_b S \mid b \\ D &\rightarrow AA, \\ E &\rightarrow BB \\ c_a &\rightarrow a, \end{aligned}$$

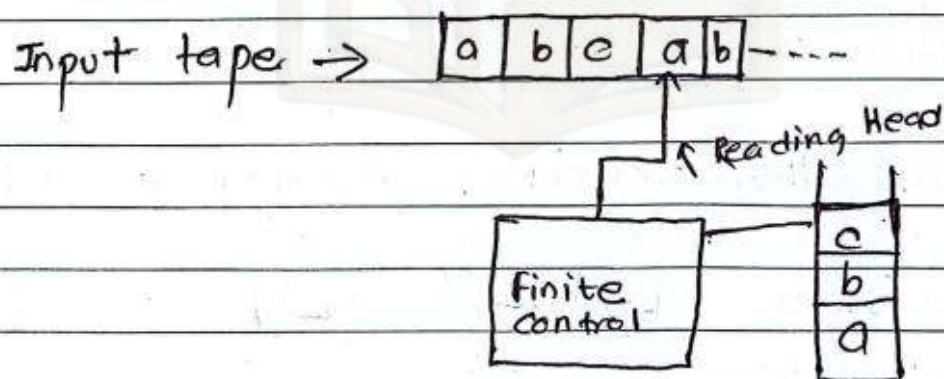
$c_b \rightarrow b$ which is the equivalent CNF

Greibach Normal Form (GNF)

GNF is of the form $A \rightarrow a\alpha$, where A is a non-terminal and a is exactly one terminal and α is the string of zero or more non-terminals.

Pushdown Automata (PDA)

Pushdown Automata (PDA) are essentially the finite Automata (FA) with auxiliary storage element called stack. Hence PDA has control of both an input tape and a stack to store what it has read.



Formal Definition

A pushdown automata is a system which is mathematically defined as follows.

$$M = (Q, \Sigma, \Gamma, s, S, F)$$

Where,

Q : is a finite set of states

Σ : is an alphabet (finite set of input symbols)

Γ : is an alphabet (finite set of stack symbols or pushdown symbols).

δ : is a mapping or transition relation which maps.

$$(Q \times (\Sigma \cup \{\epsilon\} \times \Gamma^*)) \rightarrow \text{final subset of } Q \times \Gamma^*$$

s : is the initial state $s \in Q$

F : is the set of final states $F \subseteq Q$.

Explanation of ~~mass~~ moves of Pushdown Automata:

Given the transition function of PDA for

$$(Q \times \Sigma^* \times \Gamma^*) \rightarrow (Q \times \Gamma^*)$$

- (a) The interpolation of $((q, a, z), (p, y)) \in \delta$ (where $q, p \in Q$, a is an alphabet, z and y in Γ^*) is that PDA whenever in state q with z on the top of the stack may read 'a' from the input tape, replace z by y on the top of the stack and enter state p .

(b) To push a symbol on the ~~state~~ stack, just add it on the top of the stack. This can be achieved by the transition

$$[q, a, \epsilon], (p, a) \text{ or } (q, a, \epsilon) \rightarrow (p, a)$$

(c) Similarly, to pop a symbol is to remove it from the top of the stack. The transition $[(q, a, z), (p, \epsilon)]$ pop z from the stack.

d) PDA can also behave as do nothing machine. Just read the input from the input tape and does not make any change in the state and symbol at the stack.

$$\text{Eg:- } (p, q, z), (p, z),$$

here, PDA just read input 'a' without making any change in the state and top of the stack. Waits for another input from input tape.

Pushdown Automata (PDA)

Ex #1 Design a Pushdown automata (PDA) M that accepts a language $L = \{wcw^R : w \in \{a, b\}^*\}$

Solⁿ Let $M = (Q, \Sigma, \Gamma, \delta, s, F)$ be the required PDA

Where,

$$Q = \{S, F\}$$

$$\Sigma = \{a, b, c\}$$

$$\Gamma = \{a, b\}$$

$$F = \{F\}$$

And the transition relation δ is defined by

i) $((S, a, \epsilon), (S, a))$; push a

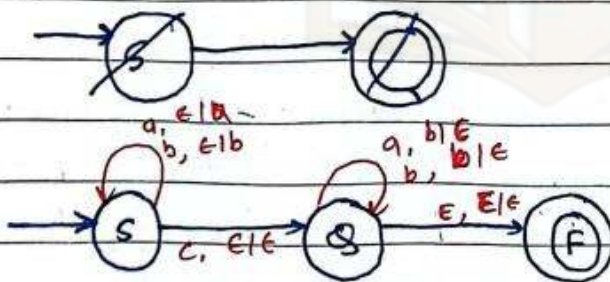
ii) $((S, b, \epsilon), (S, b))$; push b

iii) $((S, c, \epsilon), (F, \epsilon))$; Just switch the state

iv) $((F, a, a), (F, \epsilon))$; pop a

v) $((F, b, b), (F, \epsilon))$; pop b

vi) $((Q, \epsilon, \epsilon), (F, \epsilon))$; state change.



let us check for string abcba

State	Unread symbol	Stack	Transition Used
S	abcba	—	—
S	bcba	a	(i)
S	cba	ba	(ii)
Q	ba	ba	(iii)
Q	a	a	(iv)
Q	ε	ε	(v)
F	—	—	(vi)

#2 Design a pushdown automata (PDA) M that accepts a language $L = \{a^n b^n : n \geq 0\}$

soln

Let $M = (Q, \Sigma, \Gamma, \delta, S, F)$ be the required PDA

where, $Q = \{S, Q, F\}$

$\Sigma = \{a, b\}$

$\Gamma = \{a\}$

$F = \{F\}, S = S$

And the transition relation δ is defined by

i) $((S, a, \epsilon), (S, a))$, push a

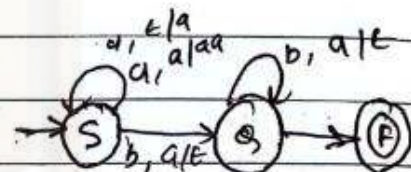
ii) $((S, a, a), (S, aa))$;

iii) $((S, b, a), (Q, \epsilon))$;

iv) $((Q, b, a), (Q, \epsilon))$;

v) $((Q, \epsilon, \epsilon), (F, \epsilon))$

vi) $((Q, \epsilon, \epsilon), (F, \epsilon))$



State	unread symbol	stack	Transition used
S	aaa bbb	-	-
S	aabbb	a	(i)
S	abbb	aa	(ii)
S	bbb	aaa	(ii)
Q	bb	aa	(iii)
Q	b	a	(iv)
Q	ε	ε	(iv)
F	ε	ε	(v)

Pushdown Automata and CFL

PDA can recognize only language for which there exists a CFL. That is the class of language accepted by PDA is exactly the class of context free language.

Construction of PDA equivalent to a CFL.

Let $G = (V_n, V_t, P, S)$ be a CFL. We must construct a pushdown automata P such that $L(P) = L(G)$.

The machine we construct has only two states, P and q and remains permanently in state q after its first move. Also, P uses V_n , the set of non-terminals and V_t the set of terminals, as its state alphabet. Now we assume

$$P = (Q, \Sigma, \Gamma, \delta, F)$$

$$\text{where } Q = \{P, q\}, \Sigma = V_t$$

$$\Gamma = V_n \cup V_t \quad (\text{i.e. set of non-terminals})$$

$$S = P$$

And the transition relation δ is defined as follows

i) $((P, \epsilon, \epsilon), (q, S))$ (as S is ~~strong~~ starting non terminal of CFL)

ii) $((q, A, A), (q, Z))$ for each rule $A \rightarrow \alpha$ in CFL

iii) $((q, a, a), (q, \epsilon))$ for each $a \in V_t$.

The PDA P begins by pushing s , the starts symbol of grammar G , on its initially empty push down storage and entering state q , using transition 1. On each subsequent step, it either replaces the top most symbol A on the stack provided that it is a non terminal by the right hand side, i.e. α of some rule $A \rightarrow \alpha$ the grammar (transition 2) OR Δ pops the top most element from the stack provided that it is a terminal the next input symbol (transition 3) OR

Example 1.

Design a PDA for the grammar $G = (V_n, V_t, P, S)$ where $V_n = \{S\}$
 $V_t = \{a, b, c\}$ and P is defined as

$$S \rightarrow asa$$

$$S \rightarrow bsb$$

$$S \rightarrow c$$

$\Rightarrow$ so^n

let the PDA be $P = (Q, \Sigma, \Gamma, S, \delta, F)$
 where,

$$Q = \{P, q\}$$

$$\Sigma = \{a, b, c\}$$

$$\Gamma = \{s, a, b, c\}$$

$$S = \{P\}$$

$$F = \{q\}$$

And the transition δ is defined as

- i) $((p, \epsilon, \epsilon), (q, s))$
- ii) $((q, \epsilon, s), (q, asa))$
- iii) $((q, \epsilon, s), (q, bsb))$
- iv) $((q, \epsilon, s), (q, c))$
- v) $((q, a, a), (q, \epsilon))$
- vi) $((q, b, b), (q, \epsilon))$
- vii) $((q, c, c), (q, \epsilon))$

Now, let us apply the transition for the string

$$w = abbcbbba$$

S.N.	State	Unread i/p	Stack	Transition us
1	p	abbcbbba	ϵ	-
2	q	abbcbbba	s	(i)
3	q	abbcbbba	asa	(ii)
4	q	bbcbba	sa	(iv)
5	q	bbcbba	bsba	(iii)
6	q	bcbba	sba	(vi)
7	q	bcbba	bsbba	(iii)
8	q	cbba	s bba	(vi)
9	q	cbba	cbba	(iv)
10	q	bba	bba	(vii)
11	q	ba	ba	(vi)
12	q	a	a	(vi)
13	q	ϵ	ϵ	-

hence
accepted

#2) Design a PDA for the CFL

$$M = (V_n, V_t, P, S)$$

$$V_n = \{S\}$$

$$V_t = \{ (,) \}$$

and P is defined as

$$S \rightarrow SS$$

$$S \rightarrow (S)$$

$$S \rightarrow \epsilon$$

consider the string $w = ()()$

solⁿ: let the PDA be $P = (Q, \Sigma, \Gamma, S, \delta, F)$

where, $Q = \{P, q\}$

$$\Sigma = \{ (,) \}$$

$$\Gamma = \{ S, \epsilon \}$$

$$S = \{P\}$$

$$F = \{q\}$$

And the transition δ is defined as:

$$i) (P, \epsilon, \epsilon) \rightarrow (q, S)$$

$$ii) (P, \epsilon, S) \rightarrow (q, SS)$$

$$iii) (P, \epsilon, S) \rightarrow (q, (S))$$

$$iv) (P, (, \epsilon) \rightarrow (q, \epsilon)$$

$$v) (q,),) \rightarrow (q, \epsilon)$$

vi)

$$i) (P, \epsilon, \epsilon) \rightarrow (q, S)$$

$$ii) (P, \epsilon, S) \rightarrow (q, SS)$$

$$iii) (P, \epsilon, S) \rightarrow (q, (S))$$

$$iv) (P, (, \epsilon) \rightarrow (q, \epsilon)$$

$$v) (q,),) \rightarrow (q, \epsilon)$$

$$vi) (q,),) \rightarrow (q, \epsilon)$$

S.N	state	Unread	Stack	Transition
1.	p	() ()	ϵ	-
2.	q	() ()	ss	(i)
3.	q	() ()	ss	(ii)
4.	q	() ()	(s)s	(iii)
5.	q	*) ()	s)s	(iv)
6.	q	) ()	)s	(vi)
7.	q	()	s	(iii)
8.	q	()	(s)	(iii)
9.	q	)	s)	(iv)
10.	q	}	)	iv
11	q	ϵ	ϵ	vi

hence Accept

Properties of context free languages

1) Closure Properties of context free language.

- CFL are closed under union, concatenation and Kleene star.

Theorem: The family of context free languages is closed under union, concatenation and Kleene star.

Proof: let L_1 and L_2 be the context-free languages generated by the following context free grammars respectively.

$$G_1 = (V_{n1}, V_{t1}, P_1, S_1) \text{ and}$$

$$G_2 = (V_{n2}, V_{t2}, P_2, S_2)$$

- Union :-

let s be a new symbol and also assume
 $G = (V_n, V_t, P, s)$ be the new grammar
 obtained from the union of G_1 and G_2
 where,

$$V_n = V_{n1} \cup V_{n2} \cup \{s\}$$

$$V_t = V_{t1} \cup V_{t2}$$

s is the start symbol and productions
 is defined as follows

$$P = \{P_1 \cup P_2 \cup \{s \rightarrow s_1 / s_2\}\}$$

here, only rules involving s are $s \rightarrow s_1$ and
 $s \rightarrow s_2$. So, G_1 and G_2 have disjoint non-
 terminals.

So, $L(G) = L(G_1) \cup L(G_2)$ is equivalent to
 saying $w \in L(G_1) \cup L(G_2)$

hence, context free languages are closed under
 union. **proved. #**

- Concatenation:

let $G = (V_n, V_t, P, s)$ be a new grammar
 obtained from concatenation (i.e. $L(G) = L(G_1) \cdot L(G_2)$)
 of G_1 and G_2 where,

$$V_n = V_{n1} \cup V_{n2} \cup \{s\}$$

$$V_t = V_{t1} \cup V_{t2}$$

$$P = P_1 \cup P_2 \cup \{s \rightarrow s_1 s_2\}$$

here, $s_1 \xRightarrow{*}_{G_1} w_1$ & $s_2 \xRightarrow{*}_{G_2} w_2$ then we can
if

claim that $s \xRightarrow{*}_G w_1 w_2$ due to the production
 $s \rightarrow s_1 s_2$.

• Kleene star:-

let $G = (V_n, V_t, P, s)$ be a new grammar, obtained
from Kleene star. (i.e. $L(G) = L(G_1)^*$) on G_1 where,

$$V_n = V_{n_1} \cup \{s\}$$

$$V_t = V_{t_1}$$

$$P = P_1 \cup \{s \rightarrow s_1 s_2, s \rightarrow \epsilon\}$$

here,

If $s_1 \xRightarrow{*}_{G_1} w$ then s gives w^* (i.e. strings
like ϵ, w_1, w_2 etc) due to production $s \rightarrow s_1 s_2$
and $s \rightarrow \epsilon$. So, we can claim in that $L(G) = L(G_1)^*$

hence, context free languages are closed
under Kleene star proved.

2. Negative Properties of context-free languages:-
- Context-free languages are not closed under
intersection and complementation.

Theorem: The family of context free languages
are not closed under intersection.

let $L = \{a^n b^n c^n : n \geq 1\}$ which is not context
free language. However, following two languages are
context free.

$$L_1 = \{a^n b^n c^m : n \geq 1, m \geq 1\}$$

$$L_2 = \{a^m b^n c^n : n \geq 1, m \geq 1\}$$

A grammar for L_1 is

$$S \rightarrow Ac$$

$$A \rightarrow aAb \mid \epsilon$$

$$c \rightarrow cc \mid \epsilon$$

Similarly, grammar for L_2 is

$$S \rightarrow AB$$

$$A \rightarrow aA \mid \epsilon$$

$$B \rightarrow bBc \mid \epsilon$$

however, the languages $L = L_1 \cap L_2 =$

$\{a^n b^n c^n : n \geq 1\}$ is not CFL. Hence, context free languages are not closed under intersection.

Proved #

Theorem:- context free languages are not closed under complementation.

Proof:-

Assume that context free languages are closed under complementation. Let L_1 and L_2 be two context free languages.

According to our assumption, $\overline{L_1}$ and $\overline{L_2}$ are also context free languages.

We know that context free languages are

also content free languages.

We know that content free languages are closed under union. So, $L_1 \cup L_2$ is also content free.

Again, from assumption, we can say that $\overline{L_1 \cup L_2}$ is also content free language.

From de-morgan's law:-

$\overline{L_1 \cup L_2} = \overline{L_1} \cap \overline{L_2}$ is content free.

which contradicts with theorem or assumption that CFL are not closed under intersection.

hence,

statement (content F.L. are not closed under complementation). **proved #**

Decision algorithm for CFLs :-

Decide whether a given CFL is empty, finite or infinite or whether a word is CFL

Emptiness :- Prove that for a given CFL, there is an algorithm to determine whether or not it can be generated any word or if the production of the form $S \rightarrow \epsilon$ or if it is empty or not.

Proof:- We have already known about nullable non-terminals and to decide the emptiness problem, we decide whether the start symbol S is nullable.

$$\text{i.e. } S \xRightarrow{*} \epsilon$$

Now, if the production is of the form $S \rightarrow a$, where 'a' is a terminal then 'a' is a word generated by CFG. Then, if there is no such production, then we follow the following algorithm.

Step 1: For each non terminal N , that has some productions of the form $N \rightarrow a$ where 'a' is a terminal a string of terminals, we choose one of them and put 'a' in the other production having N at right side, thus eliminating non-terminals N altogether.

Finiteness:-

There is an algorithm to decide whether a given CFG generates a finite or infinite language.

Proof:-

Let us assume that a grammar contains no ϵ -productions, no unit production or no useless symbols.

Assume a grammar with production of the form

$$X \xRightarrow{*} nXy$$

Since the grammar is assumed to have no ϵ production or no unit production or no useless symbols n and y can't be empty simultaneously.

Since X is neither nullable or no use-less symbol

So,

$$S \xRightarrow{*} uXv \xRightarrow{*} u^2Xv^2 \xRightarrow{*} \dots$$

where,

u, v are in V_t^* . But then,

$$S \xRightarrow{*} uXv \xRightarrow{*} u^nXv^n \xRightarrow{*} u^nXv^n \xRightarrow{*} u^nXv^n$$

possible for all n , so that $L(u)$ is infinite. But if no variable can ever repeat, then the lengths of any derivation is bounded by $|V_t|$. In that case, $L(u)$ is finite.

Step 2 Repeat Step 1 until it eliminates S or if it eliminates no new non-terminals. If S has been eliminated, then CFU produces some words, if not then not.

Example

Consider a CFU

Φ

$S \rightarrow XY$
 $X \rightarrow AX$
 $X \rightarrow AA$
 $A \rightarrow a$
 $Y \rightarrow BY$
 $Y \rightarrow BB$
 $B \rightarrow b$

soⁿ

Replace all A's by a and all B's by b.

Then, $S \rightarrow XY$
 $X \rightarrow ax$
 $X \rightarrow aa$
 $Y \rightarrow by$
 $Y \rightarrow bb$

Now, Replace X's by aa and y's by bb.
 Then, $S \rightarrow aabb$
 Replaces S by aabb

Step 2: Terminate step 1 and discover that S
 has been eliminated. So, CFN, produces
 at least one word.

The pumping lemma for CFL

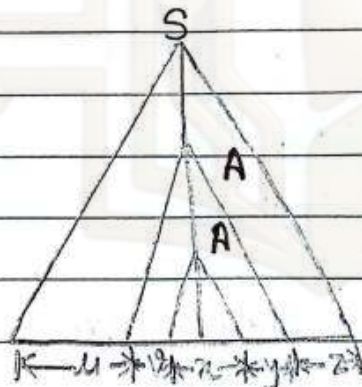
Statement:

Let G be any CFL, there is a constant ' n ', depending only on G , such that if w is in $L(G)$ and $|w| \geq n$, then we may write $w = uv^iz$ such that:

- i) $|uy| \geq 1$ that is either u or y is non-empty
- ii) $|vny| \leq n$ then for all $i \geq 0$
- iii) uv^iz is in $L(G)$

Proof:

let us take a following diagram



from above diagram, we can write

$$S \rightarrow uAz$$

$$A \rightarrow vAy$$

$$A \rightarrow x$$

By using these productions,

$$S \Rightarrow uAz$$

$$\Rightarrow uvAy$$

$$\Rightarrow uvvAyy$$

$$\Rightarrow uv^2Ay^2z$$

Show that $L = \{a^n b^n c^n : n \geq 1\}$ is not content free language.

Proof:

Let L be a content free language and w be any string in L i.e. $w \in L$ and m be pumping constant with $|w| \geq m$. Then w can be written as:

$$w = uv^2yz = a^m b^m c^m$$

Such that $|vny| \leq m$ & $|vy| \geq 1$

If the given language L is CFL, then it must satisfy $uv^i xy^i z \in L$ for $i \geq 0$

We can see that vny cannot contain both a 's and c 's as they are separated by at least $m+1$ positions.

Following cases may occur:

- i) vny contains both a 's and b 's
 Then from pumping lemma

$$w = uv^2yz = \underbrace{aa \dots aa}_{\mu} \underbrace{ab \dots bb}_{vny} \underbrace{bc \dots cc}_z$$

$$\text{let } \mu = a^p, \quad v = a^r, \quad y = b^r, \quad p+r \geq 1$$

Then for $i=2$,

$$uv^2yz = a^{m+p} b^{m+r} c^m \notin L$$

- ii) vny contains both b 's and c 's.

Then from pumping lemma

$$w = uv^2nyz = \underbrace{aa \dots aa}_{u} \underbrace{ab \dots bb}_{vny} \underbrace{bc \dots cc}_{z}$$

let $v = b^p$, $n = b^q$, $y = c^r$; $p+r \geq 1$

Then, for $i=2$

$$uv^2ny^2z = a^mb^{m+p}c^{m+r} \notin L$$

which contradicts our assumption.

hence, the given language $L = \{a^n b^n c^n : n \geq 1\}$ is not CFL.

Theorem:- The intersection of regular language with context free language is context free.

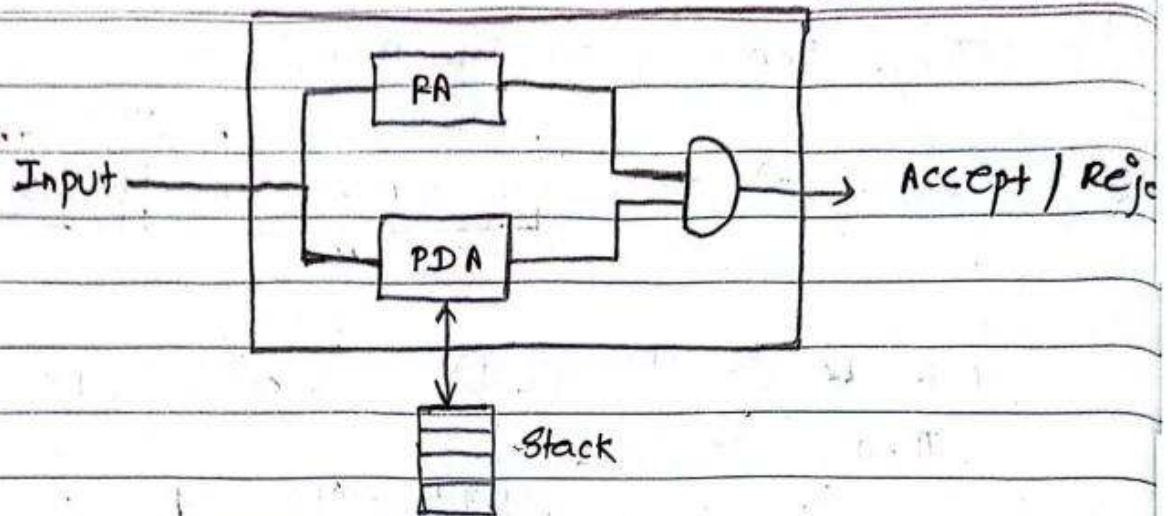
Proof: let L be a context free language and R be a regular language. Then $L = L(M_1)$ for some PDA

$M_1 = (Q_1, \Sigma_1, \Gamma_1, S_1, F_1)$ and $R = L(M_2)$ for

finite automaton $M_2 = (Q_2, \Sigma_2, S_2, S_2, F_2)$

Then, we can combine both to make new PDA

$M = (Q, \Sigma, \Gamma, S, S, F)$ as follows!



here new PDA M has following components

$$Q = Q_1 \times Q_2$$

$$\Gamma = \Gamma_1$$

$$S = (S_1, S_2)$$

$$F = F_1 \times F_2$$

And transition relation δ is defined as:

$$((p_1, p_2), w, \alpha), ((q_1, q_2), \beta)) \in \delta \text{ if and only if } ((p_1, w, \alpha), (q_1, \beta)) \in \Delta_1 \text{ and } (p_2, w) \xrightarrow{*}_{M_2} (q_2, \epsilon)$$

Here PDA M goes from state (p_1, p_2) to (q_1, q_2) in the same way that M_1 passes from state p_1 to q_1 and also keeps track that M_2 changes its state on same input.

Therefore, $w \in L(M) \iff w \in L(M_1) \cap L(M_2)$

hence, the intersection of regular language with context free language is context free

proved $\square$